Apostila de Exercícios

Orientação a Objetos

Para cada exercício resolvido, um exercício B do mesmo nível para praticar (sem resolução).
Níveis: Livro/Filme → Produto/Ingresso → Conta/Funcionário.

Material de apoio — conceitos de classes, métodos e herança

Sumário

1. Conceitos usados nestes exercícios
2. Exercício 1 — Classe Livro (com resolução)
3. Exercício 1B — Classe Filme (para você resolver)
4. Exercício 2 — Classe Produto (com resolução)
5. Exercício 2B — Classe Ingresso (para você resolver)
6. Exercício 3 — Conta Bancária (com resolução)
7. Exercício 3B — Funcionário (para você resolver)
8. Observações finais

Como usar: estude o exercício com resolução (pseudocódigo) e, em seguida, tente sozinho o exercício B do mesmo nível — sem olhar a resolução do anterior como “cola”, só como referência de forma.

1. Conceitos usados nestes exercícios

Antes das resoluções, vale lembrar os pilares aplicados nos exercícios:

- Classe: modelo que define atributos (dados) e métodos (comportamentos).
- Objeto: instância concreta de uma classe.
- Encapsulamento: controlar o acesso aos dados (ex.: getters/setters).
- Validação: o método protege o estado do objeto (estoque, saldo, percentual).
- Herança: uma subclasse reutiliza e especializa uma classe base.
- Sobrescrita: a subclasse redefine um método herdado com nova lógica.

Ordem sugerida de estudo: Livro (1 método) → Produto (vários métodos e regras) → Conta Bancária (herança e sobrescrita).

2. Exercício 1 — Classe Livro

2.1 Enunciado

Elabore uma classe Livro com os seguintes atributos:

- titulo
- autor
- anoPublicacao
- preco

E o método aplicarDesconto(), que recebe o percentual de desconto e atualiza o preço do livro.

2.2 Resolução em pseudocódigo

Classe Livro

```
CLASSE Livro
    ATRIBUTOS PRIVADOS:
        titulo       : texto
        autor        : texto
        anoPublicacao : inteiro
        preco        : numero

    CONSTRUTOR(titulo, autor, anoPublicacao, preco)
        this.titulo       <- titulo
        this.autor        <- autor
        this.anoPublicacao <- anoPublicacao
        this.preco        <- preco

    FUNCAO getTitulo()       RETORNA titulo
    FUNCAO getAutor()        RETORNA autor
    FUNCAO getAnoPublicacao() RETORNA anoPublicacao
    FUNCAO getPreco()        RETORNA preco
    PROCEDIMENTO setPreco(novoPreco)
        preco <- novoPreco

    PROCEDIMENTO aplicarDesconto(percentual)
        SE percentual < 0 OU percentual > 100 ENTAO
            ESCREVER "Percentual invalido (use 0 a 100)"
            RETORNAR
        FIM SE
        desconto <- preco * (percentual / 100)
        preco    <- preco - desconto

    FUNCAO paraTexto()
        RETORNA "Livro{titulo, autor, anoPublicacao, preco}"
FIM CLASSE
```

Programa de teste

```
INICIO
    livro1 <- NOVO Livro("O Senhor dos Aneis", "J.R.R. Tolkien", 1954, 100)
    livro2 <- NOVO Livro("1984", "George Orwell", 1949, 80)

    ESCREVER "Antes do desconto:"
    ESCREVER livro1.paraTexto()
    ESCREVER livro2.paraTexto()

    livro1.aplicarDesconto(10)   // 10%
    livro2.aplicarDesconto(20)   // 20%

    ESCREVER "Apos o desconto:"
    ESCREVER livro1.paraTexto()
    ESCREVER livro2.paraTexto()
FIM
```

Resultado esperado (exemplo)

livro1: preço 100 com 10% de desconto → 90
livro2: preço 80 com 20% de desconto → 64

3. Exercício 1B — Classe Filme

PARA VOCE RESOLVER — sem resolucao nesta apostila

Mesmo nível do Exercício 1 (Livro): uma classe, atributos simples e um método principal que altera o preço.

3.1 Enunciado

Elabore uma classe Filme com os seguintes atributos:

- titulo
- diretor
- anoLancamento
- preco

E o método aplicarDesconto(), que recebe como parâmetro o percentual de desconto e atualiza o preço do filme. Valide se o percentual está entre 0 e 100.

Crie um programa de teste que:
- Crie pelo menos dois filmes
- Mostre os dados antes do desconto
- Aplique descontos diferentes em cada filme
- Mostre os dados depois do desconto

4. Exercício 2 — Classe Produto

4.1 Enunciado

Este exercício fica entre Livro e Conta Bancária: ainda sem herança, mas com mais de um método e regras que protegem o estado do objeto.

Elabore uma classe Produto com os seguintes atributos:

- nome
- preco
- quantidadeEstoque

E os seguintes métodos:

- reporEstoque(quantidade) — aumenta a quantidade em estoque
- vender(quantidade) — diminui o estoque (não pode vender mais do que existe)
- aplicarDesconto(percentual) — recebe o percentual e atualiza o preço

Crie um programa de teste que: crie pelo menos dois produtos, repoha estoque, venda unidades, aplique desconto e mostre os dados antes e depois.

4.2 Resolução em pseudocódigo

Classe Produto

```
CLASSE Produto
    ATRIBUTOS PRIVADOS:
        nome              : texto
        preco             : numero
        quantidadeEstoque  : inteiro

    CONSTRUTOR(nome, preco, quantidadeEstoque)
        this.nome             <- nome
        this.preco            <- preco
        this.quantidadeEstoque <- quantidadeEstoque

    FUNCAO getNome()             RETORNA nome
    FUNCAO getPreco()            RETORNA preco
    FUNCAO getQuantidadeEstoque() RETORNA quantidadeEstoque
    PROCEDIMENTO setPreco(novoPreco)
        preco <- novoPreco

    PROCEDIMENTO reporEstoque(quantidade)
        SE quantidade <= 0 ENTAO
            ESCREVER "Quantidade para reposicao invalida."
            RETORNAR
        FIM SE
        quantidadeEstoque <- quantidadeEstoque + quantidade
        ESCREVER "Estoque reposto. Quantidade atual: " + quantidadeEstoque

    PROCEDIMENTO vender(quantidade)
        SE quantidade <= 0 ENTAO
            ESCREVER "Quantidade para venda invalida."
            RETORNAR
        FIM SE
        SE quantidadeEstoque >= quantidade ENTAO
            quantidadeEstoque <- quantidadeEstoque - quantidade
            ESCREVER "Venda realizada. Quantidade atual: " + quantidadeEstoque
        SENAO
            ESCREVER "Estoque insuficiente."
        FIM SE

    PROCEDIMENTO aplicarDesconto(percentual)
        SE percentual < 0 OU percentual > 100 ENTAO
            ESCREVER "Percentual invalido (use 0 a 100)"
            RETORNAR
        FIM SE
        preco <- preco - preco * (percentual / 100)

    FUNCAO paraTexto()
        RETORNA "Produto{nome, preco, quantidadeEstoque}"
FIM CLASSE
```

Programa de teste

```
INICIO
    produto1 <- NOVO Produto("Caderno", 25.0, 10)
    produto2 <- NOVO Produto("Caneta", 5.0, 50)

    ESCREVER "Antes das operacoes:"
    ESCREVER produto1.paraTexto()
    ESCREVER produto2.paraTexto()

    produto1.reporEstoque(5)     // estoque 10 -> 15
    produto2.vender(10)          // estoque 50 -> 40

    produto1.aplicarDesconto(10) // preco 25 -> 22.5
    produto2.aplicarDesconto(20) // preco 5 -> 4

    produto1.vender(3)           // estoque 15 -> 12

    ESCREVER "Apos as operacoes:"
    ESCREVER produto1.paraTexto()
    ESCREVER produto2.paraTexto()
FIM
```

Por que este exercício é intermediário?

Do Livro, reaproveita a ideia de aplicarDesconto.
Da Conta Bancária, antecipa a ideia de validar uma operação (vender só se houver estoque — parecido com sacar só se houver saldo).
Ainda não usa herança: o foco é vários métodos alterando o mesmo objeto com regras.

5. Exercício 2B — Classe Ingresso

PARA VOCE RESOLVER — sem resolucao nesta apostila

Mesmo nível do Exercício 2 (Produto): uma classe com vários métodos e regras que protegem o estado (quantidade disponível e percentual válido). Sem herança.

5.1 Enunciado

Elabore uma classe Ingresso com os seguintes atributos:

- nomeEvento
- preco
- quantidadeDisponivel

E os seguintes métodos:

- disponibilizar(quantidade) — aumenta a quantidade de ingressos disponíveis
- vender(quantidade) — diminui a quantidade (não pode vender mais do que existe)
- aplicarDesconto(percentual) — recebe o percentual e atualiza o preço

Crie um programa de teste que:
- Crie pelo menos dois ingressos (dois eventos)
- Disponibilize mais unidades
- Venda algumas unidades
- Aplique desconto
- Mostre os dados antes e depois das operações

6. Exercício 3 — Conta Bancária

6.1 Enunciado

Elabore uma classe ContaBancaria com os atributos nomeCliente, numConta e saldo, e os métodos sacar (o saldo não pode ficar negativo) e depositar.

Crie também duas subclasses:

- ContaPoupanca — com taxa de juros e método para aplicar rendimento ao saldo.
- ContaEspecial — com limite e método sacar sobrescrito (permite usar o limite).

Em um programa de teste: criar contas, sacar, depositar, aplicar rendimento e mostrar os dados do cliente.

6.2 Resolução em pseudocódigo

Classe base ContaBancaria

```
CLASSE ContaBancaria
    ATRIBUTOS:
        nomeCliente : texto
        numeroConta : texto
        saldo       : numero

    CONSTRUTOR(nomeCliente, numeroConta, saldo)
        this.nomeCliente <- nomeCliente
        this.numeroConta <- numeroConta
        this.saldo       <- saldo

    // getters e setters para cada atributo

    PROCEDIMENTO depositar(valor)
        saldo <- saldo + valor

    PROCEDIMENTO sacar(valor)
        SE saldo >= valor ENTAO
            saldo <- saldo - valor
            ESCREVER "Saque realizado. Saldo atual: " + saldo
        SENAO
            ESCREVER "Saldo insuficiente."
        FIM SE
FIM CLASSE
```

Subclasse ContaPoupanca

```
CLASSE ContaPoupanca HERDA ContaBancaria
    ATRIBUTO:
        taxaJuros : numero   // percentual, ex.: 1.0 = 1%

    CONSTRUTOR(nomeCliente, numeroConta, saldo, taxaJuros)
        CHAMAR construtor da superclasse(nomeCliente, numeroConta, saldo)
        this.taxaJuros <- taxaJuros

    PROCEDIMENTO aplicarRendimento()
        rendimento <- saldo * (taxaJuros / 100)
        depositar(rendimento)
        ESCREVER "Rendimento: " + rendimento + " | Saldo: " + saldo
FIM CLASSE
```

Subclasse ContaEspecial

```
CLASSE ContaEspecial HERDA ContaBancaria
    ATRIBUTO:
        limite : numero

    CONSTRUTOR(nomeCliente, numeroConta, saldo, limite)
        CHAMAR construtor da superclasse(nomeCliente, numeroConta, saldo)
        this.limite <- limite

    // SOBRESCRITA do metodo sacar
    PROCEDIMENTO sacar(valor)
        SE (saldo + limite) >= valor ENTAO
            saldo <- saldo - valor
            ESCREVER "Saque realizado. Saldo atual: " + saldo
            // saldo pode ficar negativo ate -limite
        SENAO
            ESCREVER "Saldo insuficiente."
        FIM SE
FIM CLASSE
```

Programa de teste (lógica sugerida)

```
INICIO
    // Criar contas
    poupanca <- NOVO ContaPoupanca("Ana", "001", 1000, 1.0)
    especial <- NOVO ContaEspecial("Bruno", "002", 500, 300)

    // Sacar
    poupanca.sacar(200)     // saldo -> 800
    especial.sacar(700)     // usa limite; saldo -> -200

    // Depositar
    poupanca.depositar(50)  // saldo -> 850
    especial.depositar(100) // saldo -> -100

    // Rendimento da poupanca
    poupanca.aplicarRendimento()  // +1% sobre 850

    // Mostrar dados
    ESCREVER poupanca.nomeCliente, poupanca.numeroConta, poupanca.saldo
    ESCREVER especial.nomeCliente, especial.numeroConta, especial.saldo,
            especial.limite
FIM
```

Diferença importante no saque

Na conta comum, só é possível sacar até o valor do saldo.
Na conta especial, o cliente pode sacar até saldo + limite. Por isso o saldo pode ficar negativo, desde que não ultrapasse o limite combinado.

7. Exercício 3B — Funcionário (herança)

PARA VOCE RESOLVER — sem resolucao nesta apostila

Mesmo nível do Exercício 3 (Conta Bancária): classe base + duas subclasses, com sobrescrita de método.

7.1 Enunciado

Elabore uma classe Funcionario com os atributos nome e salarioBase, e o método calcularSalario(), que retorna o salarioBase.

Crie também duas subclasses:

- Gerente — atributo bonus; sobrescreva calcularSalario para retornar salarioBase + bonus.
- Vendedor — atributos comissao (percentual) e totalVendas; sobrescreva calcularSalario para retornar salarioBase + (totalVendas * comissao / 100).

Crie um programa de teste que:
- Crie um gerente e um vendedor
- Mostre o salário calculado de cada um
- Altere bonus ou totalVendas e mostre o novo salário
- Exiba os dados de cada funcionário

8. Observações finais

- O pseudocódigo descreve a lógica, sem sintaxe específica de Java (ou outra linguagem).
- Cada exercício com resolução tem um exercício B do mesmo nível, sem resolução — para praticar sozinho.
- 1/1B: uma classe e um método principal (preço + desconto).
- 2/2B: vários métodos e validação de quantidade/estoque.
- 3/3B: herança e sobrescrita de método.
- Na Conta, a herança evita repetir código; em ContaEspecial.sacar, o mesmo nome de método tem comportamento diferente — polimorfismo.

Fim da apostila. Estude a resolução, resolva o exercício B, e só depois passe para o código Java.